

Angular Moderno

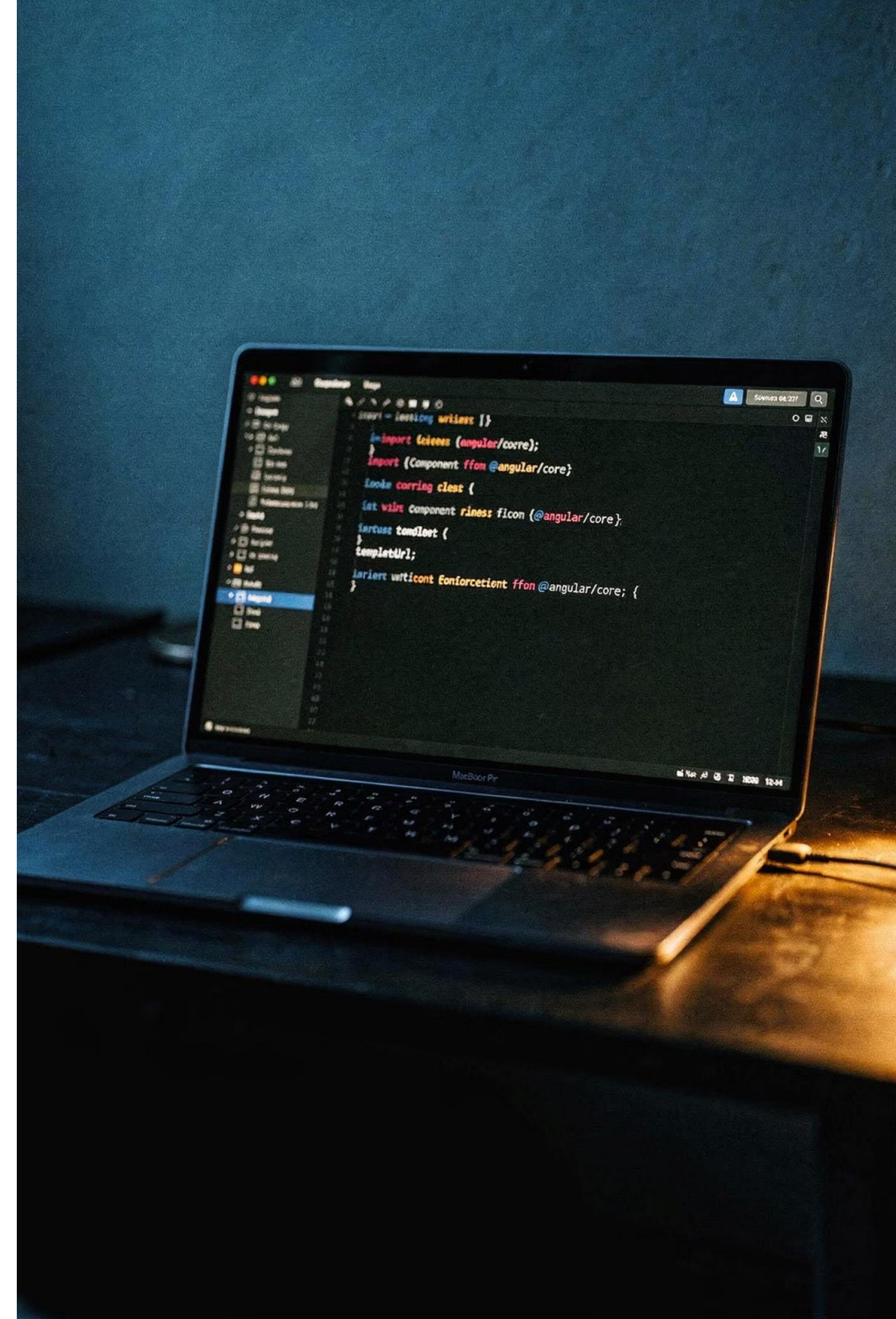
09 - Writable e Computed Signals

2026

SENAI - CRTI

09 — Writable e Computed Signals

Bem-vindo(a)! Hoje vamos evoluir o uso de Signals no nosso projeto de e-commerce, aprendendo a diferença entre **estado** (Writable Signal) e **derivação automática** (Computed Signal).



Objetivo da Aula

Ao final desta aula, você vai entender e aplicar dois conceitos centrais do Angular Moderno:

Estado Reativo

Compreender o que é um **Writable Signal** e como ele guarda e modifica o estado da aplicação com `set()` e `update()`.

Derivação Automática

Usar **Computed Signal** para derivar valores a partir de outros signals, sem cálculos manuais espalhados pelo código.

Reatividade Real

Deixar o Angular **reagir automaticamente** a mudanças de estado, eliminando código redundante e difícil de manter.

✅ Vamos parar de calcular manualmente e deixar o Angular fazer isso automaticamente.

Estado Atual do Projeto

Antes de avançar, vamos lembrar onde chegamos. O projeto de e-commerce tem a seguinte estrutura de pastas:

Estrutura de Pastas

```
src/app/  
├─ core/  
├─ shared/  
└─ features/  
    └─ produtos/  
        └─ lista-produtos/  
            ├─ lista-produtos.ts  
            └─ lista-produtos.html
```

O que já foi implementado

- `signal()` para gerenciar a lista de produtos
- Método `update()` funcionando corretamente
- Botão para adicionar um novo produto
- Template usando `produtos()` com a diretiva `@for`

Código Atual do Componente (lista-produtos.ts)

Este é o código que construímos na aula anterior (versão simplificada). Vamos partir exatamente daqui, sem alterações anteriores:

```
import { Component, signal } from '@angular/core';
import { Produto } from '../produto/produto';

@Component({
  selector: 'app-lista-produtos',
  imports: [Produto],
  templateUrl: './lista-produtos.html',
  styleUrls: ['./lista-produtos.css'],
})
export class ListaProdutos {
  produtos = signal([
    { nome: 'Notebook', preco: 3800 },
    { nome: 'Mouse', preco: 179 }
  ]);

  adicionarProduto() {
    this.produtos.update(listaAtual => [
      ...listaAtual,
      { nome: 'Teclado', preco: 250 }
    ]);
  }
}
```

O Problema Atual

O código funciona, mas ainda estamos fazendo **lógica manual desnecessária**. À medida que a aplicação cresce, isso se torna um problema real.

Exemplo do problema

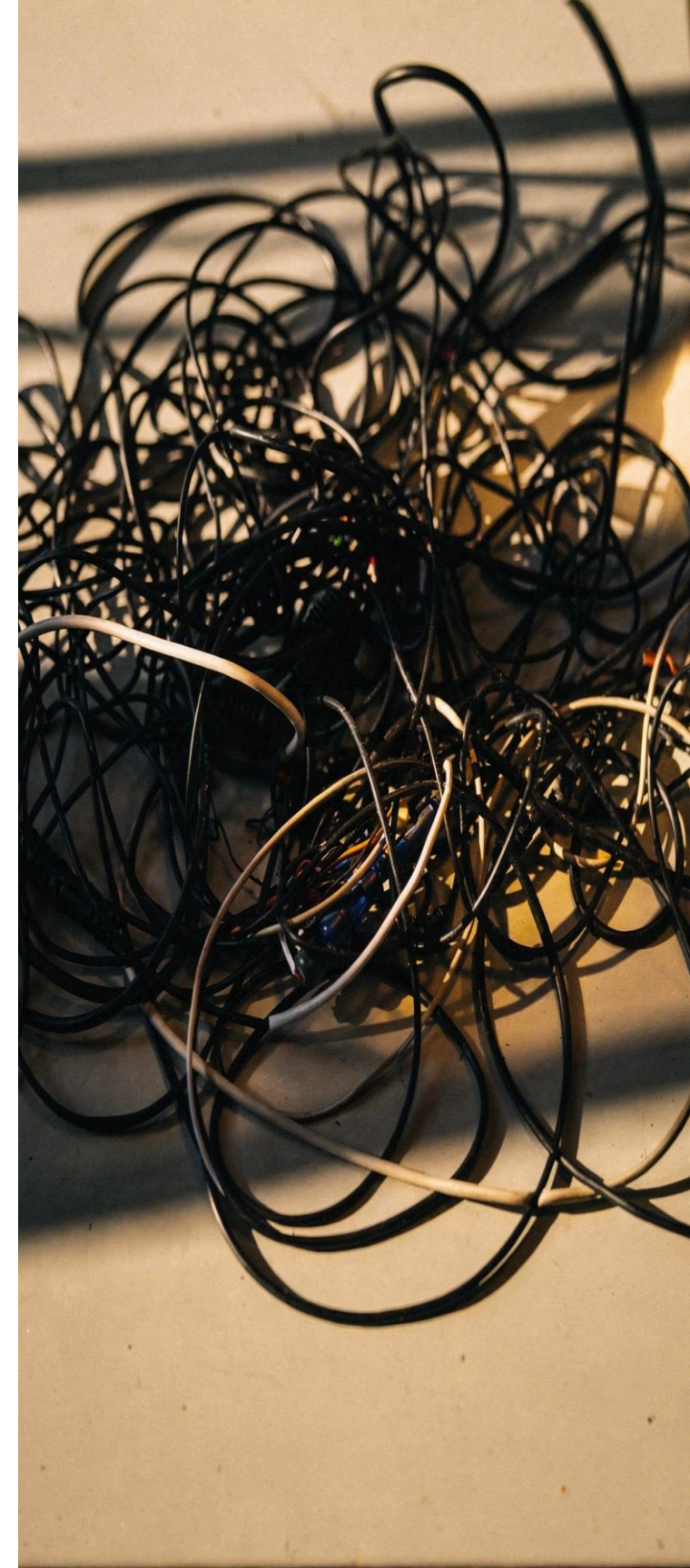
```
total = this.produtos.length; ❌
```

Se você precisar exibir o **total de produtos** ou a **soma dos preços**, precisaria calcular manualmente e garantir que os cálculos sejam atualizados a cada mudança — código extra, propenso a bugs.

Escala mal

Em aplicações reais com dezenas de signals e cálculos, manter consistência manual é **inviável**. Qualquer mudança em um estado pode quebrar outro dado não atualizado.

O objetivo de hoje é resolver exatamente esse problema usando **Computed Signals**.



Conceito 1 — Writable Signal

Um **Writable Signal** é um signal que você cria e controla. Ele guarda um valor e permite que você o modifique.

Já utilizamos isso na aula anterior!

```
produtos = signal([
  { nome: 'Notebook', preco: 3800 },
  { nome: 'Mouse', preco: 179 }
]);
```

`set(novoValor)`

Substitui completamente o valor atual do signal pelo novo valor fornecido.

`update(fn)`

Modifica o valor atual com base no estado anterior, usando uma função de transformação.

 `produtos` já é um Writable Signal — criado com `signal()`. Você tem controle total sobre ele.

Conceito 2 — Computed Signal

Um **Computed Signal** é um signal derivado de outro. Você não define seu valor, o Angular calcula automaticamente com base nos signals que ele observa.



Baseado em outro Signal

O Computed lê um ou mais signals e recalcula seu resultado sempre que qualquer um deles muda.



Somente Leitura

Você não pode usar `set()` ou `update()` em um Computed Signal. Ele é **gerenciado pelo Angular**, não por você.



Automático

Assim que o signal-fonte muda, o Computed recalcula o valor sem nenhuma intervenção manual.

⊗ REGRA FUNDAMENTAL — LEIA COM ATENÇÃO

Computed NÃO tem `set()` nem `update()`

Tentar modificar um Computed Signal manualmente causará um **erro em tempo de execução**. O Angular vai reclamar! O valor de um Computed Signal é sempre uma **consequência automática** de outros signals.

Correto

```
totalProdutos = computed(() =>  
  this.produtos().length);
```

O Angular calcula e atualiza sozinho.

Errado

```
this.totalProdutos.set(10);
```

Isso vai gerar erro! Computed é somente leitura.

Parte Prática — Passo a Passo

A seguir, vamos colocar a mão no código! Siga cada passo com calma, sem pular nenhum. Cada detalhe importa para o funcionamento correto dos Computed Signals.



Parar o Servidor de Desenvolvimento

Antes de modificar qualquer arquivo, é uma boa prática parar o servidor para evitar recarregamentos inesperados enquanto o código está incompleto.

Como parar

No terminal onde o `ng serve` está rodando, pressione:

CTRL + C

Em alguns sistemas pode ser necessário confirmar com `S` ou `Y`.

Como saber se parou?

- O terminal voltará ao prompt normal (ex: `$` ou `>`)
- O navegador vai parar de atualizar automaticamente
- Nenhuma mensagem de "compilando" será exibida

 Você pode editar os arquivos com o servidor rodando, mas parar evita surpresas durante mudanças estruturais.

Importar o `computed`

Abra o arquivo `features/produtos/lista-produtos/lista-produtos.ts` e adicione o import de `computed`:

ANTES

```
import { Component } from '@angular/core';  
import { signal } from '@angular/core';
```

DEPOIS

```
import { Component } from '@angular/core';  
import { signal } from '@angular/core';  
import { computed } from '@angular/core';
```

- ❏ O `computed` vem do mesmo pacote `@angular/core`. Apenas acrescente mais uma linha de import. Sem essa linha, o Angular não vai reconhecer a função e você terá um erro de compilação.

Criar o Computed Signal `totalProdutos`

Dentro da classe `ListaProdutos`, logo após a declaração de `produtos`, adicione:

```
totalProdutos = computed(() => this.produtos().length);
```

O que acontece aqui?

`computed()` recebe uma função. Dentro dela, chamamos `this.produtos()`. Isso faz o Angular registrar que `totalProdutos` **depende** de `produtos`.

Recálculo automático

Cada vez que `produtos` mudar (via `set()` ou `update()`), o Angular recalcula `totalProdutos` automaticamente. Você não precisa fazer nada.

Usar `totalProdutos` no Template

Abra o arquivo `lista-produtos.html` e adicione a linha abaixo para exibir o total de produtos:

```
<h2>Total de produtos: {{ totalProdutos() }}</h2>
```

⊗ ALERTA IMPORTANTE: Não esqueça os parênteses `()` após `totalProdutos`! Signals são funções. Sem os parênteses, o Angular exibirá o objeto signal em vez do valor calculado. Esse é o erro mais comum de iniciantes com Signals.

Lembre-se: todo signal, seja Writable ou Computed, precisa ser **chamado como função** no template.

Primeiro Teste — Verificar o Total

Agora vamos testar o que implementamos até aqui. Volte ao terminal e inicie o servidor:

```
ng serve
```

1 Acesse o navegador

Abra `http://localhost:4200` e verifique se a frase "Total de produtos: 2" aparece na tela (os dois produtos iniciais).

2 Clique em "Adicionar Produto"

Observe que o número atualiza automaticamente para **3** sem nenhuma lógica extra. O Computed Signal fez tudo sozinho!

3 Confirme o comportamento

Clique mais vezes e veja o total subir. Isso é a reatividade do Angular 21 em ação.

🟢 Debugging: No navegador, abra o console (F12) se não funcionar.

Criar o Computed Signal `valorTotal`

Agora vamos criar uma derivação de negócio real: a soma de todos os preços dos produtos. Adicione logo abaixo de `totalProdutos`:

```
valorTotal = computed(() => {  
  return this.produtos().reduce((total, item) => total + item.preco, 0);  
});
```

O que é `reduce()`?

É um método de array do JavaScript que percorre cada item acumulando um resultado.

Aqui, começa do zero (0) e vai somando o campo `preco` de cada produto.

Por que usar Computed aqui?

Porque a soma **depende diretamente** da lista de produtos.

Sempre que um produto for adicionado, o `valorTotal` **recalculará automaticamente**, sem nenhum código extra.

Exibir **valorTotal** no Template

Abra o arquivo `lista-produtos.html` e adicione a linha abaixo do total de produtos:

```
<h3>Valor total: R$ {{ valorTotal() }}</h3>
```

Seu template deve estar parecido com isso agora:

```
@for (item of produtos(); track item.nome) {  
  <app-produto  
    [nome]="item.nome"  
    [preco]="item.preco"  
    (produtoSelecioneado)="exibirProduto($event)"  
  ></app-produto>  
}  
<br>  
<button (click)="adicionarProduto()">Adicionar Produto</button>  
  
<h2>Total de produtos: {{ totalProdutos() }}</h2>  
<h3>Valor total: R$ {{ valorTotal() }}</h3>
```

 Não esqueça os parênteses em `valorTotal()`! A mesma regra dos outros signals se aplica aqui.

Teste Completo – Tudo Junto

Com o servidor rodando (`ng serve`), realize o teste completo:

1

Estado Inicial

Ao carregar: **2 produtos**, total de **R\$ 3.979** (Notebook + Mouse).

2

Após 1 clique

Teclado adicionado: **3 produtos**, total de **R\$ 4.229**. Ambos atualizaram!

3

Após mais cliques

Total e valor continuam aumentando automaticamente. Nenhuma linha de código extra foi necessária.

- ✔ Se os dois valores atualizam automaticamente ao clicar no botão, parabéns!
Os Computed Signals estão funcionando corretamente!

Diferença entre `set()` e `update()`

Vamos criar um novo método para entender na prática quando usar cada um. Adicione ao componente:

```
substituirProdutos() {  
  this.produtos.set([{ nome: 'Produto novo', preco: 999 }]);  
}
```

`set()` — Substitui Tudo

Ignora completamente o estado anterior. A lista inteira é **trocada** pelo novo valor. Use quando quiser reiniciar ou redefinir o estado.

`update()` — Modifica com Base no Atual

Recebe o valor atual como parâmetro e retorna o novo valor. Use quando precisar **adicionar, remover ou transformar** o estado existente.

Adicionar Botão "Substituir Lista" no Template

Abra o arquivo `lista-produtos.html` e adicione um segundo botão para testar o `set()`:

```
<p><button (click)="adicionarProduto()">Adicionar Produto</button></p>
```

```
<p><button (click)="substituirProdutos()">Substituir Lista</button></p>
```

- ❏ Note que os botões têm funções completamente diferentes: um **adiciona** à lista existente (`update`), o outro **substitui** a lista inteira (`set`). Ambos, porém, fazem os Computed Signals recalcularem automaticamente.

Teste Final – Substituição de Lista

Com o servidor rodando, faça o teste final do botão "Substituir Lista":

- 1 Adicione alguns produtos
Clique em "Adicionar Produto" algumas vezes para ter 4 ou 5 itens na lista com total e valor acumulados.
- 2 Clique em "Substituir Lista"
Observe que a lista inteira desaparece e é substituída por apenas **1 produto** ("Produto novo", R\$ 999).
- 3 Verifique os Computed Signals
Total de produtos volta para **1** e valor total para **R\$ 999**.
Os Computed Signals reagiram automaticamente ao `set()`!

✔ Tanto `set()` quanto `update()` disparam o recálculo dos Computed Signals. Isso é a reatividade do Angular funcionando!



Consolidando os Conceitos

Vamos fixar a diferença fundamental entre os dois tipos de signals que vimos hoje:

Writable Signal — "Guarda Estado"

Criado com `signal()`. Você controla o valor diretamente com `set()` e `update()`. É a **fonte da verdade** da sua aplicação. Exemplo: lista de produtos, usuário logado, filtro ativo.

Computed Signal — "Calcula Estado"

Criado com `computed()`. Você **não controla** o valor — ele é calculado automaticamente. Nunca use `set()` ou `update()`. Exemplo: total de itens, soma de preços, lista filtrada.

 Pense assim: Writable Signal é o **dado bruto**. Computed Signal é o **dado processado**.

Erros Comuns — Fique Atento!

Estes são os erros que iniciantes cometem com mais frequência ao trabalhar com Signals.

Leia com atenção e evite-os no seu código:

1. Esquecer os `()` no template

`{{ totalProdutos }}` exibe o objeto, não o valor. Sempre use `{{ totalProdutos() }}`.

2. Tentar usar `set()` em Computed

`this.totalProdutos.set(10)` gera erro em tempo de execução. Computed é somente leitura.

3. Fazer cálculo fora do `computed()`

Calcular no construtor ou em um método não reativo não atualiza automaticamente. Sempre use `computed()` para valores derivados.

4. Misturar variável comum com signal

`total = 0` não é reativo. Mudanças nessa variável não atualizam o template automaticamente.

5. Sobrescrever o signal com atribuição direta

`this.produtos = [...novaLista]` não funciona. Use sempre `set()` ou `update()`.

Comparação Didática — Writable vs Computed

Uma visão lado a lado para fixar as diferenças entre os dois tipos de Signals:

Característica	Writable Signal	Computed Signal
Como criar	<code>signal(valorInicial)</code>	<code>computed(() => ...)</code>
Quem define o valor	Você (programador)	O Angular (automático)
Usa <code>set()</code>	✅ Sim	❌ Nunca
Usa <code>update()</code>	✅ Sim	❌ Nunca
Atualiza automaticamente	Não (você atualiza)	✅ Sempre
Exemplo no projeto	<code>produtos</code>	<code>totalProdutos</code> , <code>valorTotal</code>
Uso ideal	Estado da aplicação	Derivações e cálculos

Conexão com a Aula Anterior

Esta aula foi uma evolução direta do que construímos na aula anterior. Veja como o conhecimento se acumulou:

Antes — Estado com Signals

Criamos o estado reativo com `signal()`, implementamos `update()` para adicionar produtos e exibimos a lista com `@for` no template.

1

2

3

Evolução do Projeto

O componente `lista-produtos` agora é totalmente reativo: estado gerenciado por Writable Signals e cálculos automatizados por Computed Signals.

Agora — Derivação Automática

Evoluímos para `computed()`, criando `totalProdutos` e `valorTotal` que se atualizam sozinhos. Também aprendemos a diferença entre `set()` e `update()`.

✔ Você passou de "gerenciar estado" para "derivar dados automaticamente" — uma grande evolução no modo de pensar Angular Moderno!

Resultado Esperado – O Que Você Conquistou

Ao concluir esta aula, você domina um dos conceitos mais poderosos do Angular Moderno:



Estado Gerenciado

Usar `signal()` como fonte da verdade, modificando com `set()` e `update()` corretamente.



Derivação Automática

Criar `computed()` que recalculam sozinhos, sem necessidade de lógica manual espalhada pelo código.



Código Limpo

Aplicação mais previsível, organizada e fácil de manter. Pilares do desenvolvimento profissional com Angular 21+.

Lembre-se: **Writable** guarda. **Computed** calcula. **Effect()** reage. Esses três são os pilares dos Signals no Angular Moderno.

Dúvidas

?

